

Balancing QoS and power consumption in programmable 5G infrastructures

Alessandro Carrega | Giancarlo Portomauro | Matteo Repetto  | Giorgio Robino

S3ITI National Lab, CNIT, Genova, Italy

Correspondence

Matteo Repetto, S3ITI National Lab, CNIT,
16145 Genova, Italy.

Email: matteo.repetto@cnit.it

Funding information

European Commission H2020 under the
projects ARCADIA and MATILDA,
Grant/Award Number: 645372 and 761898

Abstract

The advent of fifth generation is promising to push far more intelligence than today to the network boundary, hence boosting novel computing models based on fog/edge paradigms. The need for proximity in computation, coupled with various forms of mobility, will be responsible for dynamic shifting of workload within the system, with large fluctuations in resource usage. This eventually turns into poor energy efficiency of the whole infrastructure. However, improving efficiency usually deteriorates quality of service, hence the dilemma about how to balance these two contrasting aspects.

In this paper, we propose a framework that leverages the increasing programmability of ICT infrastructures to pursue a linear relationship between power consumption and workload, while safeguarding quality of service. Our approach is based on workload consolidation and extensions to existing cloud management software. We collected both real measurements from an experimental testbed and performance analysis from simulations to evaluate the consolidation strategy in more complex environments.

1 | INTRODUCTION

The underpinning concept behind fifth generation (5G) is a radical transformation of communication networks, ie, beyond mere performance increase (latency, bandwidth, reliability, and density), 5G envisions the installation of capillary computing and storage resources. All this infrastructure builds a rich and pervasive information and communications technology (ICT) fabric, which turns networks into intelligent orchestration platforms.^{1,2} Different from the cloud, where resources are located in one or a few big data centers, 5G infrastructures will consist of large distributed, pervasive, heterogeneous, and multidomain environments, with both traditional data centers and a large number of small/tiny edge installations. This ultimately complicates management and poses new challenges for deployment and orchestration.

The 5G infrastructures will be specifically designed to fulfill the challenging requirements of many vertical industries, ie, smart manufacturing, energy, e-health, automotive, media, and entertainment.² With the grow of user-centric services, characterized by interactivity and strict constraints on latency, the workload is expected to follow user distribution and mobility patterns, hence leading to uneven, unsteady, and nonuniform distribution within the network. Consequently, the usage of peripheral installations is expected to dynamically vary with hourly, daily, weekly, and seasonal periodicity.

According to current management practice in telecommunication networks, the whole system is expected to be largely overdimensioned. In fact, breaking ongoing services is considered a severe quality detriment, and it is far less tolerable than denying set-up of new ones. This immediately turns into low resource usage and large energy waste, which means an unfair and inefficient balance between the two quantities.

Running idle devices is the main cause of the nonlinear relationship between power consumption and resource usage.³ Unfortunately, a recent analysis points out that relying on low-power idle technologies will not be the most effective strategy.⁴ The inefficiency raises very fast in 5G infrastructures, amplified by the large number of edge installations. Since energy efficiency is listed among the 5G key performance indexes, it is essential to make extensive usage of advanced power saving mechanisms both in communication and computing subsystems, without jeopardizing quality of service (QoS) parameters like availability, reliability, communication bandwidth, and CPU time.

In this paper, we describe our framework for balancing Energy Efficiency (EE) with QoS. The framework includes a consolidation algorithm and an energy-efficient infrastructure. The consolidation algorithm optimizes the usage of physical resources; the infrastructure consists of extensions to existing cloud management software (CMS) for real-time monitoring of power, software-defined control of the network, and management of power states of devices. The major novelty of our framework is the association of QoS constraints to each virtual machine (VM), which allows to combine fast and effective resource provisioning and bandwidth allocation with the highest level of power efficiency. In addition, our work represents the first experimental implementation of consolidation in a mixed cloud/software-defined network (SDN) environment, explicitly targeting edge computing installations in 5G infrastructures.

This paper extends our initial work,⁵ which describes the power monitoring infrastructure integrated in OpenStack. The current framework provides a number of additional modules (consolidation, power management, and software-defined networking), which together build a complete ecosystem and application of the original work. In addition, we report additional power measurements collected over a larger set of devices.

This paper is organized as follows. We review related work in Section 2. We introduce the motivation and the overall description of our framework in Section 3. We describe the implementation of the energy-efficient infrastructure and the consolidation algorithm in Section 4 and Section 5, respectively. We report evaluation and numerical analysis from both the testbed and simulations in Section 6. Finally, we give our conclusions and plans for future work in Section 7.

2 | RELATED WORK

Efficient usage of IT equipment tackles the well-known nonlinearity between performance and power consumption of both computing and networking devices.⁶ Beside improving hardware efficiency, other mechanisms have been proposed to modulate the power consumption of single devices, including voltage and frequency scaling, low-power idle, and stand-by states.^{3,7-9} In a cloud environment, where many devices are present and resources are virtualized, consolidation has proven to be the most effective strategy to exploit such mechanisms, allowing to minimize the total consumption of the entire infrastructure in a coordinated way.^{7,8}

Workload consolidation is one of the most efficient ways to save energy in data centers. It implies gathering VMs on the minimum number of servers, without violating their resource requirements (CPU, RAM, and network bandwidth), so as to shut down the remaining infrastructure. Many algorithms have been proposed for this purpose, which consider the power consumption of servers,^{10,11} network infrastructures,^{12,13} a combination of both,¹⁴⁻¹⁶ and even cooling systems.¹⁷

The consolidation problem has often been modeled as a modified *bin packing* problem, seeking the optimal operating point between energy and performance.¹⁸ However, VM placement is not exactly equivalent to a three-dimensional (CPU, RAM, and IO) bin packing problem, and some anomalies could arise from this approach and similar ones in the literature.¹⁹ Given the NP-completeness of the problem, most authors end up proposing heuristics to find a suboptimal solution in acceptable time.^{18,19}

Several researchers have tackled the trade-off between traffic routing and power consumption in data center networks. Heller et al¹² modeled the consolidation of network flows as a Multi-Commodity Flow Problem (MCFP) and a greedy bin packing. In other cases, VMs are explicitly placed so as to minimize the network power consumption. Shirayanagi et al¹⁵ targeted fault tolerance and took into consideration placement of backup VMs.

The problem of consolidation has also been modeled by jointly considering the power consumption of servers and network equipment. Kliazovich et al²⁰ defined a composite metric-based on a hierarchical model that accounts for the energy consumption of servers and network equipment, which is used to find the best balance between server utilization and queue size.

Many papers focus on the mathematical formulation of the problem and on heuristic solving methods to circumvent its NP-completeness. However, the real effectiveness in practice is highly influenced by the correct power model of each single device because using the smallest possible number of servers does not guarantee to achieve the overall minimal energy consumption.¹⁸ As first approximation, few heavy loaded servers consume less power than many lightly loaded servers,

due to the nonnegligible energy drawn in idle state. Building on this rule of thumb, many consolidation algorithms use simple linear or stepwise models to describe the trade-off between CPU usage and power consumption, since CPU is the main source of power consumption.²¹ However, these models cannot take into account the complexity brought by the presence of multiple CPUs, multiple cores, power management features (voltage/frequency scaling, low-power idle, etc.).²¹ Furthermore, servers from different vendors have different characteristics, and even the behavior of the same server changes while aging.²² Statistical models have been proposed for both CPUs and network cards, which better capture the dynamics of real hardware;²³ however, they are complex to manage and to be properly tuned for different devices. These considerations motivated our previous work, which extended the OpenStack Ceilometer component with power measurements. The latter can then be correlated to the current CPU utilization ratio to describe the energy profile of each single device.^{5,24}

Tight integration between cloud technologies and SDN have been debated for long, since it was demonstrated that networking performance has a significant impact on the quality of Cloud services.²⁵ In light of the expected convergence, the hottest topic has been interconnection of distributed clouds and points of presence through wide area networks, with specific focus on on-demand provisioning with guaranteed service level agreements (SLAs) and QoS.^{26–29} In many cases, network programmability is only used as enabler for some form of virtualization and abstraction, based on overlays, to steer traffic across functions and services.^{30,31} Indeed, turning high-level connectivity policies into proper network configuration is not trivial because it falls in the class of NP-complete problems; only few attempts have been made in this direction.^{32,33}

Looser integration is available within data centers, ie, CMS provides networking-as-a-service functions by creating virtual instances of L2 switches, routers, firewalls, etc. (eg, in OpenStack Neutron and VMware NSX). Networking-as-a-service usually provides overlay links for each tenant and requires manual and static configuration of the underlying physical infrastructure (eg, virtual local area networks (VLANs), provider networks, routing, and QoS paths); plugins to automatically configure such devices have only been designed for a few vendors (eg, OpenStack ML2 plugins for Brocade and Nexus devices). No QoS mechanisms are therefore available to guarantee bandwidth, latency, and jitters, apart rate limiters and bandwidth shapers in the hypervisor. The OpenDayLight plugin already brings SDN capabilities to OpenStack, but it just provides a different configuration interface to virtual switches in hypervisors and does not manage the physical infrastructure. Indeed, QoS must be integrated within all edge and cloud installations to fully support the stringent key performance indexes settled by 5G networks.³⁴ In addition, CMS should take advantage of programmable networks to configure end-to-end paths with strict QoS constraints.³⁵ This is clearly necessary to support any network slicing mechanism.

3 | QUALITY OF SERVICE VERSUS ENERGY EFFICIENCY IN EDGE COMPUTING

Fifth-generation networks target vertical industries, which challenging requirements cannot be satisfied by the cloud (eg, geographical distribution, computing proximity, and transmission latency).² The overall quality of the service is determined by several parameters, ie, communication latency and bandwidth, CPU time available for mobile services, acceptance rate of new connections, seamless handovers/migrations, and resilience to failures. Such parameters are more or less directly affected by transmission and computing mechanisms (eg, priority classes and queue management for network packets and scheduling and preemption for software jobs), placement decisions (eg, number of communication hops), and operational decisions (eg, rerouting of packet streams, live migrations of applications and services, and resource allocation strategies); we briefly indicate these aspects under the generic term of “QoS management.” Effective QoS management is essential to guarantee the required performance levels during the whole service lifetime,³⁶ so as to effectively support smart manufacturing, e-health, energy, automotive, and media and entertainment industries.

Telecom operators have been facing QoS for long by deploying far more resources than strictly needed to ward off congestion from their networks. Even if QoS mechanisms were systematically used within the system, overdimensioning would still be necessary for radio access segments and their backhubs, where mobility results into uneven, unsteady, and nonuniform distribution of users, sometimes without fixed periodicity patterns. Since overdimensioning leads to poor utilization of the overall system, it raises power consumption and crumbles the overall efficiency, and it is no more acceptable today in its current form, both for environmental and economical reasons.

Current power-saving mechanisms can provide an effective solution to increase the power efficiency of overdimensioned 5G networks, whenever properly integrated into virtualization and management practice.³⁷ As a matter of fact, the active workload in each point of presence can be consolidated on the smallest number of servers, while putting idle

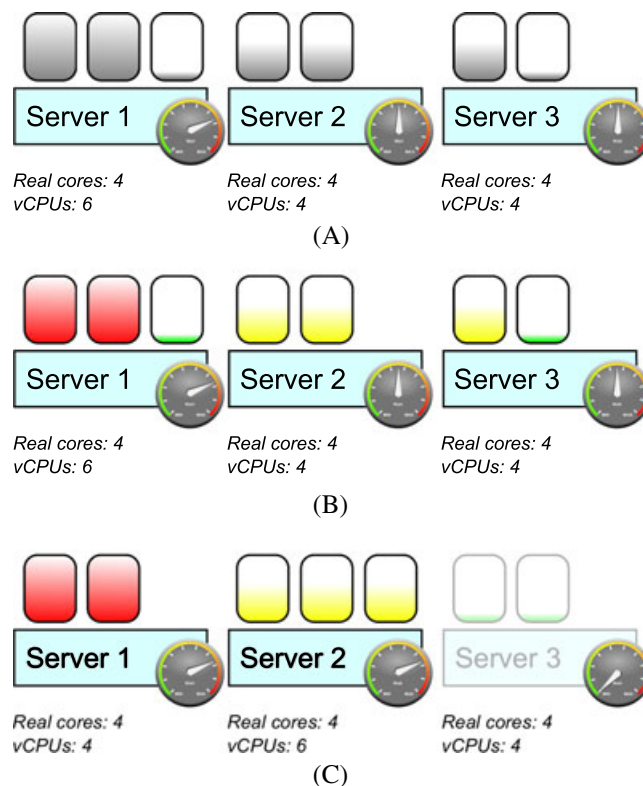


FIGURE 1 Consolidation in business-as-usual scenario and the proposed model. A: Business-as-usual; B: QoS-EE model; C: QoS-EE model, with consolidation and power saving

devices in deep-sleep modes (technically speaking, suspend-to-ram or suspend-to-disk). We even dare to state that energy efficiency may improve QoS while boosting new business models. We originally proposed this concept for the cloud,³⁸ and we now describe its implementation for 5G networks in this paper.

To better illustrate the potential trade-off between QoS and energy efficiency let us consider the following pure infrastructure-as-a-service (IaaS) scenario. Reliability and high availability often require to deploy additional software instances of an application, which are used in case of sudden peaks of requests, failure of the primary components, and movement of users to a different location.³⁹ Provisioning such instances in real time is not currently possible (though unikernel technologies leave this possibility open for the future) because of technical and administrative issues. Indicatively, it usually takes from minutes to days to have a VM ready. Hence, the alternative is proactive provisioning, which implies to pay for idle instances that might even be never used. The main cause of this counterproductive situation is due to CMS that does not know what each user instance is meant for; this could be largely improved if this information was available. The idea is to reward the users that explicitly identify temporary idle and unused virtual resources by lower fares, compensated by larger power saving for infrastructure providers.

Figure 1 introduces our energy efficiency paradigm with a practical example in a very simple IaaS scenario. Let us consider three servers with 4 cores each. Let us represent the current usage level of each VM by the extension of the internal coloration. We have 8 VMs, requesting 2 vCPUs each; we suppose that two of them are just preprovisioned for some performance reason. In the business-as-usual scenario (Figure 1A), VMs can be consolidated on a small number of servers by overcommitting physical resources. However, since some VMs are preprovisioned and may be used only occasionally, this behavior is likely to alternatively result in low server utilization and energy wasting (when preprovisioned VMs are not used) and violation of SLAs (when all VMs are used). Without any specific information about utilization and criticality of each VM, the infrastructure provider behaves in a blind way and must consider all resources fully active; this means that the users have to pay for the two additional machines even if they will never use them.

Now, let us suppose we know QoS requirements for each VM (Figure 1B). We use a very simple model, where a limited number of different colors (red, yellow, and green) correspond to different QoS constraints. For example, we can set

- *red*, for mission-critical applications, which require strict allocation of CPU, memory, and bandwidth requirements;
- *yellow*, for working instances that tolerate some degree of overcommitment on CPU and bandwidth; and
- *green*, for unused and idle instances, which only require to be available within a maximum delay.

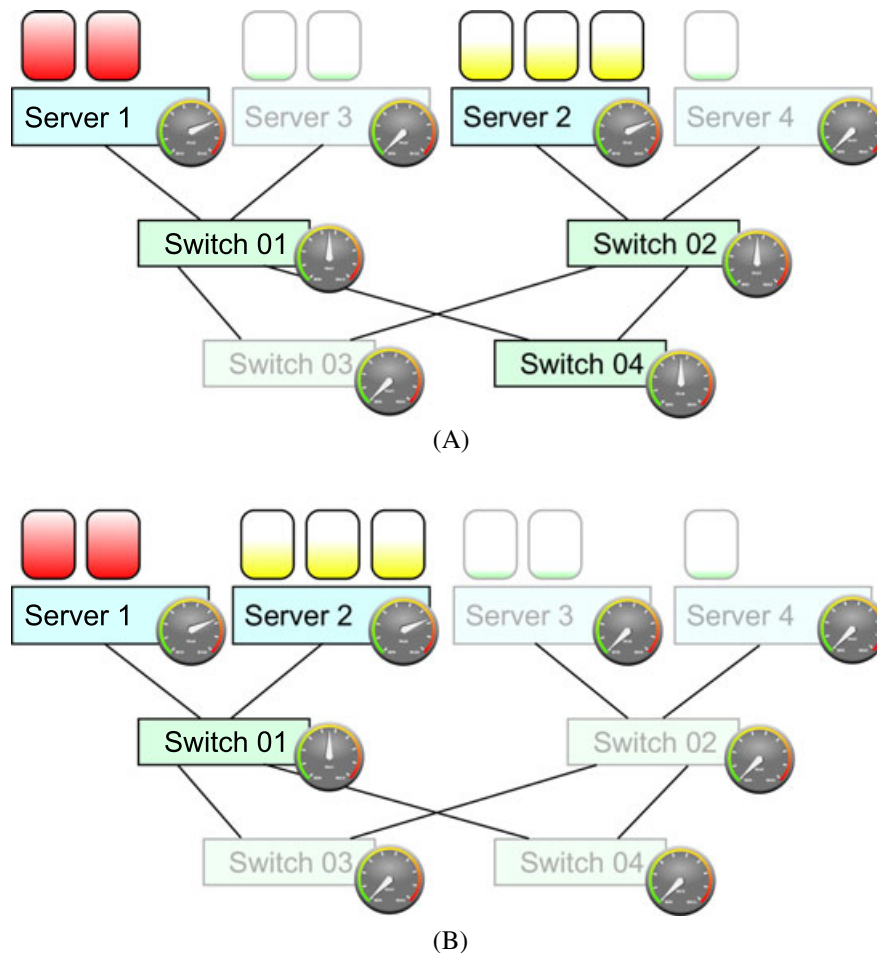


FIGURE 2 Simple network architecture, which may represent a single pod in a three-tier architecture with full bisectional bandwidth. Virtual links between virtual machines are not shown for simplicity. A: Network not involved in the consolidation procedure; B: Network involved in the consolidation procedure

Based on this model, we can modify the consolidation strategy so as to cluster together red/yellow instances (without breaking QoS constraints), while green instances are left aside and do not need to be consolidated on the smallest number of servers (Figure 1C). Since we know that green VMs are not used, we can put each server hosting this kind of VMs into low-power mode (suspend-to-ram) and quickly* resume it when one of its VMs changes to the yellow/red status.

The trade-off between QoS and energy efficiency is not limited to computing servers but also largely depends on the communication infrastructure. Consolidation is a dynamic process that moves virtual resources around the data center. Slicing in 5G will need proper mapping between high-level policies and the physical network, well beyond basic overlays used today in many cloud installations. Both processes require dynamicity and autonomicity from the underlying communication infrastructure, which can only be achieved by exploiting the increasing level of programmability. We see a further possibility to improve the overall efficiency in this scenario, ie, by also applying power modulation and dynamic power states to network equipment. The first step consists in taking bandwidth constraints into account in the consolidation process; this avoids binding too many virtual networks over the same physical links. The second step consists in choosing active servers for consolidation so as to minimize the number of necessary network switches (see Figure 2); all devices that are not connected to any active equipment (server or switch) can then be put in low-power mode.

To implement the concept described so far, we need two main components as follows:

- an energy-efficient infrastructure, which supports power management, VM migration, and network programmability (Section 4) and
- a consolidation algorithm, which computes optimal placement of VMs by taking into account QoS requirements on CPU and bandwidth (Section 5).

*Current commercial hardware can resume from suspend-to-ram (advanced configuration and power interface (ACPI) S3 state) in a few hundreds of milliseconds.

Next sections describe in details these two components, their innovation to current state of the art, and the mix of measurements and simulations we have carried out so far.

4 | ENERGY-EFFICIENT INFRASTRUCTURE

Our implementation of an energy-efficient infrastructure that fits the overall concept described in Section 3 includes the following elements:

- CMS that implements the IaaS model;
- SDN, fully programmable for optimizing network paths according to bandwidth requests;
- monitoring, which collects measurements about power consumption and CPU utilization from all servers and network devices; and
- management interfaces, which allow interaction both for using and managing the infrastructure.

Figure 3 shows the architecture of our energy-efficient infrastructure, which can be used for each point of presence in a 5G/Network Function Virtualization domain. It provides IaaS and is designed with two kinds of interfaces, ie, North and East. The North interface is conceived for users and provides a traditional application programming interface (API) enhanced with QoS properties. The East interface is used for control and management (ie, to retrieve data, apply configurations, and trigger power-saving mechanisms), allowing the integration of pluggable logics. We have already developed the software for workload consolidation; it may also be used to provide additional functions (eg, to expose power/performance relationship, as described in the work of Bolla et al³⁸) and to interwork with the rest of the 5G infrastructure.

4.1 | Cloud management software

We use the open-source OpenStack framework as CMS. Our installation includes authentication (Keystone), computing (Nova), networking (Neutron), image (Glance), storage (Cinder), and telemetry (Ceilometer) modules. The Neutron ML2 plugin is used for virtual networking, together with OpenvSwitch software-based switch. The VLAN encapsulation creates independent tenant networks over the physical infrastructure. Our choice is motivated by the fact that VLAN is the only network virtualization technology that can be managed by OpenFlow in physical switches. The network node is installed on a dedicated machine. A shared file system is available on all compute nodes, and live-migration of VMs is possible between compute nodes.

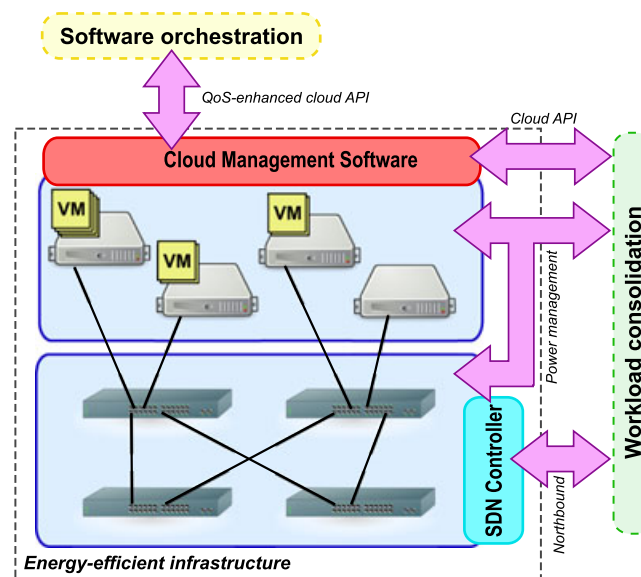


FIGURE 3 Architecture of the energy-efficient infrastructure. Dotted components represent additional components we used in this paper to demonstrate the trade-off between power consumption and energy efficiency. API: Application Programming Interface; SDN: Software-Defined Network; QoS: Quality Of Service; VM: Virtual Machine

4.2 | Software-defined network

The software-defined network is composed of OpenFlow software switches, running OpenvSwitch. We use software switches running on low-end desktops because hardware equipment usually does not provide ACPI power states. In addition to OpenFlow, network devices also expose the GAL interface (*Green Abstraction Layer*⁴⁰), a recently introduced standard to report and modify energy-related characteristics (power states, power/performance relationship, etc.). The GAL is used to collect the power profile of each device (eg, the description of how energy consumption changes with different utilization levels) and to change the power state from active (ACPI S0) to sleep (ACPI S3). Devices are resumed back to full operation (from ACPI S3 to S0) by the wake-on-LAN protocol.

OpenDayLight is used as network controller. We use the l2switch feature to provide full connectivity among all nodes, while explicit flows are configured between VMs with bandwidth requirements. Configuration of QoS parameters (priority queues, traffic shapers, and scheduling disciplines) along the switch is not implemented yet, but it is already on the development roadmap.

4.3 | Monitoring framework

The main objective for the monitoring component is to collect measurements from heterogeneous devices. It retrieves data from different meters, with different communication protocols, and with different data formats. To this aim, we use Kwapi,⁴¹ a modular framework for monitoring power consumption and publishing data in OpenStack Ceilometer. We extended Kwapi to collect data about CPU usage of the hypervisors, so we can build the power profile of each host.⁵

Kwapi includes drivers to query commodity power meters over simple network management protocol, serial/usb wattmeters, and the intelligent platform management[†] interface available in many recent computing boards; this covers most hardware deployed in existing data centers, including legacy servers with a single motherboard and blade servers.

4.4 | North and East interfaces

The North interface is the OpenStack API. Currently, QoS support in OpenStack is very limited and only covers a few configurations in the hypervisor. We enrich the existing interface with more semantics for our purposes, in the following way.

- *active/paused* states: VMs can be in different steady states,[‡] roughly corresponding to ACPI power states. We make use of the PAUSED state to indicate *green* VMs and ACTIVE for *red/yellow* ones. Virtual machines in PAUSED state will not be used until a new state transition; hence, hypervisors that only host PAUSED VMs can sleep.
- *metadata*, which are properties in the form $\langle key, value \rangle$ associated to each VM and stored in the metadata server.[§] These metadata are used to distinguish between yellow/red VMs and to set bandwidth requirements between VMs.

The extended semantics cannot be used by the plain OpenStack installation but can be retrieved by external management software via the East interface (cloud API in administration mode).

The East interface is a collection of control and management APIs to exploit the unique features of this infrastructure. From the top of the right side of Figure 3, we find the cloud API, the power management interface, and the Northbound interface of the network controller. We expect that the enhanced North interface be directly used by software orchestration tools. It is easy to include QoS characteristics in software development models like TOSCA⁴² and ETSI network function virtualization.⁴³ Specifically, such extensions are already included in the ARCADIA context model,[¶] which extends TOSCA, as already discussed in our previous work.³⁸

The *cloud API* is the standard OpenStack interface, used to retrieve information about the virtualization system (running VMs, state, tenant networks, VLAN ids, etc.) and to trigger migration of VMs. It is also used to detach hypervisors from the OpenStack controller before putting them to sleep, so as to avoid any unreachability error.

[†]The intelligent platform management interface provides management and monitoring capabilities of computer systems over the network, without support from the firmware or the operating system.

[‡]OpenStack documentation — Server states. URL: https://developer.openstack.org/api-guide/compute/server_concepts.html.

[§]Metadata can be inserted when the VM is created, but can also be updated later. See: <https://wiki.openstack.org/wiki/NovaImageCreationAPI>.

[¶]<http://www.arcadia-framework.eu/documentation/context-model/>

The *power management* interface provides access to the monitoring framework described in Section 4.3. In our case, it is used by consolidation algorithms to know how the power consumption changes in different load conditions (CPU usage), to put servers and software switches in sleep state, and to resume them, as previously described.

Finally, the *Northbound* interface is used to program the network infrastructure. We use RESTCONF,⁴⁴ available as OpenDayLight feature, which provides high-level network abstraction through the Yang model. This interface is used to set flows in the underlying datapath, according to the optimization strategy computed by the consolidation algorithm.

5 | WORKLOAD CONSOLIDATION

We consider an edge computing installation like a small data center, made of physical servers, network devices, and links. We explicitly address an IaaS model, ie, every application consists of one or more software components, each deployed in a separate VM, which represents the granularity of our proposal. Virtual machines are created and destroyed according to the application life-cycle management logic, which is not relevant for the model and corresponds to an arrival/departure process. Virtual machines are classified into different groups, according to QoS constraints (eg, color labels).

5.1 | Data center infrastructure model

We model a data center as a weighted directed graph $\mathcal{G}(\mathbb{M}, \mathbb{E})$, where $\mathbb{M}$ and $\mathbb{E}$ are sets of nodes and edges, respectively. In the graph $\mathcal{G}$, a node $m \in \mathbb{M}$ is either a server (leafs) or network device (roots and internal nodes); a directed edge $e_{m_1}^{m_2} \in \mathbb{E}$ is a network link from node m_1 to m_2 with bandwidth $b_{m_1}^{m_2}$, where $m_1, m_2 \in \mathbb{M}$. With $z_{m_1}^{m_2} \in \{0,1\}$ we indicate if the edge $e_{m_1}^{m_2} \in \mathbb{E}$ is active (1) or not (0). To simplify the description of the proposed model, we consider directed edges with $e_{m_1}^{m_2} \neq e_{m_2}^{m_1}$ for $m_1, m_2 \in \mathbb{M}$.

We denote with $\mathbb{S}$ and $\mathbb{N}$ the sets of servers and network devices, respectively ($\mathbb{M} = \mathbb{S} \cup \mathbb{N}$). Considering that the number of ports can vary from one network device to another, we use σ_n to denote the number of ports into the network device $n \in \mathbb{N}$. The resources of a server $s \in \mathbb{S}$ are defined by a pair made of the number of CPUs[#] C_{cpu}^s and the quantity of RAM C_{ram}^s , in MB.

Finally, we define the sets $\mathbb{D} := \{d_{s_1}^{s_2} | s_1, s_2 \in \mathbb{S}\}$ and $\mathbb{L} := \{l_{s_1}^{s_2} | s_1, s_2 \in \mathbb{S}\}$ composed of the minimum number of hops $d_{s_1}^{s_2}$ and the residual bandwidth $l_{s_1}^{s_2}$ between servers, respectively.

Table 1 summarizes the notation for the data center infrastructure model (we use $\mathcal{R}_{\geq 0}$ and $\mathcal{Z}_{\geq 0}$ for the set of nonnegative real and integer numbers, respectively).

5.2 | Virtual machine model

User applications are modeled as sets of VMs, including communication patterns between each pair. The model includes resource (CPU, RAM, and link bandwidth) and QoS/SLA requirements.

We define $\mathbb{V}$ as the set of running VMs. Each VM v is characterized by requirements on CPUs(Ψ_{cpu}^v) and RAM (Ψ_{ram}^v); additionally, we define $\Psi_{\text{bw}}^{(v_1, v_2)}$ as the bandwidth (in Mbps) required between VMs v_1 and v_2 . To take into account the dynamic nature of the arrival/departure process, we define the start and duration time of a VM with Ψ_{start}^v and Ψ_{dur}^v , respectively.

For QoS/SLA, we define a set of groups $\mathbb{G} \equiv \{\text{red}, \text{yellow}, \text{green}\}$. We use the binary variable δ_v^g to indicate whether the VM $v \in \mathbb{V}$ belongs to the group $g \in \mathbb{G}$ or not. The variable ξ_{cpu}^v represents the fraction of CPU guaranteed to VM $v \in \mathbb{V}$, and it is set to ξ_{cpu}^g , based on group membership.

The VM placement is expressed with x_v^s , which indicates whether the VM $v \in \mathbb{V}$ is placed on the server $s \in \mathbb{S}$ or not. The bandwidth reserved on each link $e_{m_1}^{m_2} \in \mathbb{E}$ is indicated as $y_{(v_1, v_2)}^{(m_1, m_2)}$. The distance d_M^g indicates for each group the maximum number of communication hops between VMs that must not be exceeded by the placement algorithm.

Table 2 summarizes the notation for the VM model.

[#] For the sake of simplicity, in this paper, the term CPU refers to the minimum processing unit that can be set in a low-power mode.

TABLE 1 Data center infrastructure model notation

Parameter	Description
$\mathbb{M}$	Set of nodes (servers or network devices).
$m \in \mathbb{M}$	Node (server or network device).
$\mathbb{E}$	Set of directed edges.
$e_{m_1}^{m_2} \neq e_{m_2}^{m_1} \in \mathbb{E}$	Directed edge (network link) from node m_1 to m_2 .
$b_{m_1}^{m_2} \in \mathcal{R} \geq 0$	Bandwidth of network link between nodes m_1 and m_2 .
$z_{m_1}^{m_2} \in \{0, 1\}$	Indicates if the edge between nodes m_1 and m_2 is active (1) or not (0).
$\mathbb{S}$	Set of servers.
$s \in \mathbb{S}$	Server.
$C_{\text{cpu}}^s \in \mathcal{R} \geq 0$	Number of CPUs available in the server s .
$C_{\text{ram}}^s \in \mathcal{R} \geq 0$	Quantity of RAM (in MB) available in the server s .
$d_{s_1}^{s_2} \in \mathcal{Z} \geq 0$	Minimum number of hops between servers s_1 and s_2 .
$\mathbb{D} := \{d_{s_1}^{s_2} s_1, s_2 \in \mathbb{S}\}$	Set of minimum number of hops between servers s_1 and s_2 .
$l_{s_1}^{s_2} \in \mathcal{R} \geq 0$	Residual bandwidth between servers s_1 and s_2 .
$\mathbb{L} := \{l_{s_1}^{s_2} s_1, s_2 \in \mathbb{S}\}$	Set of residual bandwidth between servers s_1 and s_2 .
$\mathbb{N}$	Set of network devices.
$n \in \mathbb{N}$	Network device.
$\sigma_n \in \mathcal{Z} \geq 0$	Number for ports available in network device n .

TABLE 2 Virtual machine model notation

Parameter	Description
T	Simulation duration.
$\mathbb{R}$	Set of incoming request in the interval $[0, T]$.
$r \in \mathbb{R}$	Incoming request consisting a in new set of VMs.
$\mathbb{V}^{(r)}$	Set of VMs of request r .
$\mathbb{V}$	Set of VMs.
$v \in \mathbb{V} \cup \mathbb{V}^{(r)}$	Virtual Machine (VM).
$x_v^s \in \{0, 1\}$	Indicates if VM v is placed on server s .
$y_{(v_1, v_1)}^{(m_1, m_2)} \in \mathcal{R}_{\geq 0}$	Amount of bandwidth from VMs v_1 to v_1 assigned to link $e_{m_1}^{m_2}$.
$\Psi_{\text{cpu}}^v \in \mathcal{R}_{\geq 0}$	Number of CPUs required by the VM v .
$\Psi_{\text{ram}}^v \in \mathcal{R}_{\geq 0}$	Amount of RAM (in MB) required by the VM v .
$\Psi_{\text{start}}^v \in [0, T]$	Arrival time (in second) of the VM v .
$\Psi_{\text{dur}}^v \in [0, T]$	Duration time (in second) of the VM v .
$\Psi_{\text{bw}}^{(v_1, v_2)} \in \mathcal{R}_{\geq 0}$	Bandwidth (in Mbps) required for the communications between VMs v_1 and v_2 .
$\xi_{\text{cpu}}^v \in [0, 1]$	Fraction of CPU guaranteed to VM v .
$\mathbb{G}$	Set of groups (<i>red</i> , <i>yellow</i> , and <i>green</i>).
$g \in \mathbb{G}$	Group.
$\delta_v^g \in \{0, 1\}$	Indicates if VM v belongs to group g .
$\xi_{\text{cpu}}^g \in [0, 1]$	Fraction of CPU guaranteed to the VMs belonging to group g .
d_M^g	Maximum number of hops between the VMs belonging to group g .

5.3 | Request arrival and departure process

Requests from tenants are generated randomly; the arrival and departure processes follow an exponential distribution.⁴⁵

The generic request r by a tenant consists of a new set of VMs $\mathbb{V}^{(r)}$. The request $r \in \mathbb{R}$ arrives and has a duration in the interval $[0, T]$; different duration times can be generated for each VM.

TABLE 3 Power model notation

Parameter	Description
$\Phi_m(U_m)$	Power consumption of m .
Φ_m°	Idle power consumption of m .
Φ_m^*	Maximum power consumption of m .
U_m	Utilization level of m .

5.4 | Power model

We consider a multicomponent utilization-based power model, which defines the power consumption of a data center according to the resource utilization of servers and network devices. According to the typical nonlinear behavior of energy consumption for ICT devices,⁴⁶ we define the power consumption of a device (server or switch) as a function of resource utilization U_m . The latter represents the percentage of CPU usage for servers and the fraction of active ports for network devices. As a result, the total power consumption $\Phi_m(U_m)$ of device $m \in \mathbb{M}$ can be computed as

$$\Phi_m(U_m) = \Phi_m^\circ + U_m \cdot \Phi_m^*, \quad \forall m \in \mathbb{M}, \quad (1)$$

where Φ_m° represents the idle power (when no operation is being carried out) and Φ_m^* is the additional power consumption at full load.

Table 3 summarizes the notation for the power consumption model.

5.5 | Consolidation algorithm

Roughly speaking, our target is the design of a consolidation strategy that:

1. keeps *red* VMs on the smallest number of servers, while assuring enough resources are provisioned to meet their stringent QoS requirements;
2. groups *yellow* VMs together on the smallest number of servers, even on the same servers as *green* VMs if enough resources are available;
3. distributes *green* components among all servers, with no stringent constraints on grouping the largest number of them together, since servers hosting only *green* components will sleep; and
4. places VMs according to the total power consumption of active servers, network devices, and network links to minimize the power consumption of the overall data center.

Given the complexity and the dimension of the problem, we divided the consolidation procedure into two steps, according to similar works in the literature,^{19,47,48} ie, placement and routing.

The first step (placement) allocates the VMs to the servers, with the objective to minimize their total power consumption under resource constraints. The second step (routing) computes optimal routes in the network, with the objective to minimize the total power consumption of network devices under bandwidth and link capacity constraints.

The pseudocode for the consolidation procedure is shown in Algorithm 1. The strategy starts by removing the terminated VMs (line 5) and by computing the residual bandwidth between servers (line 6). The *placement* function (line 8) takes the current set of VMs $\mathbb{V}$ and the set of requests $\mathbb{V}^{(r)}$ and returns a new allocation plan $\mathbb{V}^*$. This plan contains the placement of VMs on servers; if a solution was not found, the returned set is empty $-\emptyset$. The *routing* step (line 10) is only called when a solution was found at the previous step; it updates the allocation plan $\mathbb{V}^*$ with bandwidth allocation on each link. Like in the previous case, if the solution was not found, the returned set is empty. Finally, if a solution for the overall problem was found (line 11), it is returned (line 14); otherwise, the consolidation is tried again after removing some VMs from the request set $\mathbb{V}^{(r)}$ (line 12). The whole procedure is repeated until either a solution is found or all the incoming VMs are discarded (lines 7 to 14). In this case, the algorithm returns the original plan (line 15) and no changes occur.

According to the description in Algorithm 1, the consolidation procedure must be run every time a user submits a new request $\mathbb{V}^{(r)}$. However, some (or many) VMs may terminate between consecutive arrivals, leading to suboptimal allocation. For this reason, the consolidation procedure is also run periodically with an empty request set, whenever the elapsed time since the last consolidation exceeds a given time interval θ_{migr} .

Algorithm 1 Consolidation**Require:**

- 1: sets of allocated VMs $\mathbb{V}$ and requested VMs $\mathbb{V}^{(r)}$
- 2: sets of servers $\mathbb{S}$, network machines $\mathbb{N}$, and edges $\mathbb{E}$
- 3: sets of groups $\mathbb{G}$ sets of minimum hops between servers $\mathbb{D}$

```

4: function CONSOLIDATION( $\mathbb{V}, \mathbb{V}^{(r)}, \mathbb{S}, \mathbb{N}, \mathbb{E}, \mathbb{G}, \mathbb{D}$ )
5:    $\mathbb{V} \leftarrow \text{REMOVEDTERMINATEDVM}(\mathbb{V})$ 
6:    $\mathbb{L} \leftarrow \text{COMPUTERESIDUALBANDWIDTH}(\mathbb{V}, \mathbb{S}, \mathbb{N}, \mathbb{E})$ 
7:   while  $\mathbb{V}^{(r)} \neq \emptyset$  do
8:      $\mathbb{V}^* \leftarrow \text{PLACEMENT}(\mathbb{V}, \mathbb{V}^{(r)}, \mathbb{S}, \mathbb{G}, \mathbb{D}, \mathbb{L})$ 
9:     if  $\mathbb{V}^* \neq \emptyset$  then
10:       $\mathbb{V}^* \leftarrow \text{ROUTING}(\mathbb{V}, \mathbb{V}^*, \mathbb{S}, \mathbb{N}, \mathbb{E})$ 
11:     if  $\mathbb{V}^* = \emptyset$  then
12:       $\mathbb{V}^{(r)} \leftarrow \text{CUTREQUEST}(\mathbb{V}^{(r)}, \mathbb{G})$ 
13:     else
14:       return  $\mathbb{V}^*$ 
15: return  $\mathbb{V}$ 

```

In the following, we give mathematical details about placement and routing functions.

1. *Placement algorithm*: This function solves an optimization problem for placing VMs on the available servers. It returns the updated set of VMs $\mathbb{V}^*$, which contains the binding of VMs to servers; if no solution is found, the empty set $\emptyset$ is returned.

We formulate the placement problem as integer linear programming. The problem minimizes the total power consumption of servers (2), under resource constraints that keep into account the limited availability of CPUs and RAM. The formal definition of the optimization problem is reported in (2) to (8)

$$\min \sum_{\substack{\{x_v^s\} \\ v \in \mathbb{V}^{(r)} \\ s \in \mathbb{S}}} \Phi_s(U_s) \quad (2)$$

$$x_v^s \in \{0, 1\}, \forall s \in \mathbb{S}, \quad \forall v \in \mathbb{V}^{(r)} \sum_{s \in \mathbb{S}} x_v^s = 1, \quad \forall v \in \mathbb{V}^{(r)} \quad (3)$$

$$\sum_{v \in \mathbb{V}^{(r)}} x_v^s \cdot \Psi_{\text{cpu}}^v \cdot \tilde{\xi}_{\text{cpu}}^v \leq C_{\text{cpu}}^s \sum_{v \in \mathbb{V}} x_v^s \cdot \Psi_{\text{cpu}}^v \cdot \tilde{\xi}_{\text{cpu}}^v, \quad \forall s \in \mathbb{S} \quad (4)$$

$$\sum_{v \in \mathbb{V}^{(r)}} x_v^s \cdot \Psi_{\text{ram}}^v \leq C_{\text{ram}}^s - \sum_{v \in \mathbb{V}} x_v^s \cdot \Psi_{\text{ram}}^v, \quad \forall s \in \mathbb{S} \quad (5)$$

$$\delta_{v_1}^g \cdot \delta_{v_2}^g \cdot \sum_{s_1, s_2 \in \mathbb{S}} (x_{v_1}^{s_1} \cdot x_{v_2}^{s_2} \cdot d_{s_1}^{s_2}) \leq d_M^g, \quad \forall v_1, v_2 \in \mathbb{V}^{(r)}, \quad \forall g \in \mathbb{G} \quad (6)$$

$$x_{v_1}^{s_1} \cdot x_{v_2}^{s_2} \cdot \Psi_{\text{bw}}^{(v_1, v_2)} \leq l_{s_1}^{s_2}, \quad \forall v_1, v_2 \in \mathbb{V}^{(r)}, \quad \forall s_1, s_2 \in \mathbb{S} \quad (7)$$

$$U_s = \sum_{v \in \mathbb{V}^{(r)}} \frac{x_v^s \cdot \Psi_{\text{cpu}}^v \cdot \tilde{\xi}_{\text{cpu}}^v}{C_{\text{cpu}}^s} + \sum_{v \in \mathbb{V}} \frac{x_v^s \cdot \Psi_{\text{cpu}}^v \cdot \tilde{\xi}_{\text{cpu}}^v}{C_{\text{cpu}}^s}, \quad \forall s \in \mathbb{S}. \quad (8)$$

The optimization function (2) consists in the minimization of the power consumption with respect to the variable x_v^s , where $v \in \mathbb{V}^{(r)}$ and $s \in \mathbb{S}$; (3) to (8) represent the constraints.

The first constraint (3) guarantees that each VM is only placed on one server. The next two constraints ensure that the VMs are placed on servers with sufficient residual resources (CPU and RAM). We consider overcommitment, so only a fraction $\tilde{\xi}_{\text{cpu}}^v$ of the requested CPU for each VM $v \in \mathbb{V}$ is guaranteed. The constraint in (6) sets the maximum distance in number of hops between VMs. The constraint in (7) assures that the residual bandwidth between the servers where the VMs will be allocated is compatible with the request bandwidth. Finally, the last equation is used to compute the utilization level of the server $s \in \mathbb{S}$ as a fraction of the CPU utilized by the VMs.

2. *Routing algorithm*: This function returns the updated sets of VMs, now including the paths in the network and bandwidth allocation on each link. If a solution is not found, the empty set $\emptyset$ is returned.

We formulate the routing problem as linear programming. The objective is to minimize the total power consumption of network devices, under constraints on the bandwidth requirements between VMs and the link capacities. The formal definition of the optimization problem is reported in (9) to (15)

$$\min_{\substack{\{y_{(v_1,v_2)}^{(m_1,m_2)}\} \\ v_1,v_2 \in \mathbb{V}^{(r)} \\ e_{m_1}^{m_2} \in \mathbb{E}}} \sum_{n \in \mathbb{N}} \Phi_n(U_n) \quad (9)$$

$$y_{(v_1,v_2)}^{(m_1,m_2)} \geq 0, \quad \forall v_1, v_2 \in \mathbb{V}^{(r)}, \quad \forall e_{m_1}^{m_2} \in \mathbb{E} \quad (10)$$

$$\sum_{v_1,v_2 \in \mathbb{V}^{(r)}} y_{(v_1,v_2)}^{(m_1,m_2)} \leq b_{m_1}^{m_2} - \sum_{v_1,v_2 \in \mathbb{V}} y_{(v_1,v_2)}^{(m_1,m_2)}, \quad \forall e_{m_1}^{m_2} \in \mathbb{E} \quad (11)$$

$$\sum_{\substack{e_n^s \in \mathbb{E} \\ n \in \mathbb{N}}} x_{v_2}^s \cdot y_{(v_1,v_2)}^{(n,s)} = \Psi_{bw}^{(v_1,v_2)} \cdot |x_{v_1}^s - x_{v_2}^s|, \quad \forall v_1, v_2 \in \mathbb{V}^{(r)}, \quad \forall s \in \mathbb{S} \quad (12)$$

$$\sum_{\substack{e_n^s \in \mathbb{E} \\ n \in \mathbb{N}}} x_{v_1}^s \cdot y_{(v_1,v_2)}^{(s,n)} = \Psi_{bw}^{(v_1,v_2)} \cdot |x_{v_1}^s - x_{v_2}^s|, \quad \forall v_1, v_2 \in \mathbb{V}^{(r)}, \quad \forall s \in \mathbb{S} \quad (13)$$

$$\sum_{\substack{e_n^m \in \mathbb{E} \\ m \in \mathbb{M}}} y_{(v_1,v_2)}^{(m,n)} = \sum_{\substack{e_n^m \in \mathbb{E} \\ m \in \mathbb{M}}} y_{(v_1,v_2)}^{(n,m)}, \quad \forall v_1, v_2 \in \mathbb{V}^{(r)}, \quad \forall n \in \mathbb{N} \quad (14)$$

$$U_n = \frac{1}{\sigma_n} \cdot \sum_{\substack{e_n^m \in \mathbb{E} \\ m \in \mathbb{M}}} \text{sgn}(z_m^n + z_n^m), \quad \forall n \in \mathbb{N}. \quad (15)$$

The optimization function (9) consists in the minimization of the power consumption with respect to the variable $y_{(v_1,v_2)}^{(m_1,m_2)}$, where $v_1, v_2 \in \mathbb{V}^{(r)}$ and $e_{m_1}^{m_2} \in \mathbb{E}$; (10) to (15) represent the constraints.

The first constraint forces the nonnegativity of the variable $y_{(v_1,v_2)}^{(m_1,m_2)}$. The second one requires that the total bandwidth to allocate is less or equal to the link capacity minus the bandwidth of the VMs already allocated. The next three constraints force the demand satisfaction ((12) and (13)) and the flow conservation (14). Finally, (15) is used to compute the utilization level of the network machine $n \in \mathbb{N}$ as a fraction of the active ports.

3. *CutRequest algorithm*: Since the total amount of resources requested by the users may exceed the system capacity, we need an admission control to avoid that excessive overcommitment lead to violation of QoS and SLA.

Our simple scheme just discards VMs according to their group and demand in terms of resources. To this aim, we assign an increasing priority to the groups in $\mathbb{G}$ according to the following order:

- *green*,
- *yellow*, and
- *red*.

We first select the VMs from $\mathbb{V}^{(r)}$ that belong to the group with the lowest order. Among them, we select VMs with the highest number of required CPUs. If more than one candidate is still present, we choose the ones with the greatest required amount of RAM. Finally, if the selected VMs are still more than one, we randomly discard one of them.

6 | EVALUATION AND NUMERICAL RESULTS

We carried out functional and performance evaluation over a mix of real testbed and simulations. The real testbed has been deployed primary for demonstration of the feasibility and performance measurements, while simulations provide extended evaluation of the consolidation algorithm over more meaningful size and topology.

6.1 | Experimental setup

We deployed a working testbed according to the architecture shown in Figure 3. It is a small testbed, but it is rather representative of edge installations, which are not expected to have the same size as modern data centers. It is composed of the following:

- three compute nodes (Intel NUC6i7KYK, i7-6770HG 2.60GHz 4 cores, 4 GB RAM);

- four software switches that interconnect compute and network nodes (data network^{||}), running on a heterogeneous set of low-end commercial desktops (Atom, Pentium, and Phenom; 2 to 4 cores; around 1.1GHz);
- OpenStack controller, network node, and OpenDayLight controller, deployed as VMs on a high-end server; and
- management network on a separate infrastructure.

The installed software is the Newton release for OpenStack and Carbon for OpenDayLight. For software orchestration, we used bash scripts for simple tests. The consolidation algorithm is implemented in the optaplanner framework.

6.2 | Functional evaluation

We ran functional tests on the experimental infrastructure. The specific objective was to verify the correct behavior of all operations needed for consolidation. Specifically, we investigated the following issues.

- Live migration of VMs. Virtual machines are correctly moved between different servers, in a seamless and transparent way for users, which means no service disruption was noted. We measured no packet losses for migration of a VM while pinging another host at the fastest allowed rate (scheduling one ECHO request message every 1 ms). We are well aware that heavy memory usage patterns delay the migration and may cause longer service disruption, but these conditions usually correspond to tight QoS constraints (ie, *red* VMs) and we do not want to migrate in this situation (as discussed in Section 3).
- Power consumption and CPU usage are correctly gathered by the monitoring framework. Results are reported in Section 6.4.
- Software-defined network. The l2switch feature in OpenDayLight provides connectivity to all compute nodes. Our extension correctly retrieves the VLAN ID for each tenant network from OpenStack and configures dedicated flows according to the paths computed by the consolidation algorithm.
- Power saving. We successfully put both compute nodes and switches in sleep mode and resume them, without breaking the overall infrastructure operation. Sleeping compute nodes are seen as DETACHED, but they return immediately available as soon as they are resumed. Sleeping switches are not temporarily seen by the OpenDayLight controller, but they keep the correct OpenFlow configuration and restart forwarding packets when resumed.

6.3 | Workload consolidation

In this section, we validate the effectiveness of the proposed algorithm using simulation and numerical results. We show the efficiency of VM placement and routing by using the proposed consolidation algorithm and another strategy for comparison. We also assess the impact of migration in terms the power, energy-saving, and resource overhead.

1. *Metrics*: We consider the following metrics for performance evaluation:

- *Acceptance rate*, the average percentage of accepted VM requests and
- *Energy savings*, the average savings compared with the case with a simpler allocation strategy.

2. *Scenarios*: We consider a simulation time of $T = 1000$ units; a new request can arrive at any time between 0 and 1000.

Similar to the work of Dalvandi et al,⁴⁵ we use an exponential distribution for the arrival rate of incoming requests and a truncated log-normal distribution for the number of VMs per request and the mean request duration.⁴⁵ To simulate variable conditions, we dynamically change the mean arrival rate of requests μ_{arr}^r as follows: 5, 1, and 10 per time unit. At the beginning of the simulation, μ_{arr}^r is equal to 5 (medium load). Then, in the middle of the simulation, it is decreased to 1 (low load). Finally, at the end of the simulation, it is incremented to 10 (high load). Similarly, we change the average number of VMs per incoming request μ_{vm}^r for each corresponding value of μ_{arr}^r as follows: 2 (medium load), 1 (low load), and 3 (high load). Random values are generated in the interval [1,5]. The mean request duration has average value of 5 and is generated in the interval [1,10] time units.

We also generate randomly the resource requests according to a truncated log-normal distribution with mean values $\Psi_{cpu}^{(\mu)} = 1$, $\Psi_{ram}^{(\mu)} = 1$ GB, and $\Psi_{bw}^{(\mu)} = 10$ Mbps, in the intervals [1, 2], [0.5, 2] GB, and [0, 20] Mbps, respectively. The group membership is assigned based on the uniform distribution probability (red: 20%, yellow: 50%, and green: 30%).

^{||} The communication infrastructure for hosted VMs is denoted as “guest” network in latest OpenStack releases.

3. **Numerical results:** We wrote an event simulator in MATLAB to evaluate the proposed algorithm. We validated the effectiveness of our energy-aware (EA) algorithm through a comparison with a simpler consolidation strategy (Matching Pursuit, MP), which does not take into account the group membership and the power consumption of the infrastructure. To better analyze the results, we consider the combination of the load levels defined in the previous section for the mean request arrival rate and the average number of VMs per request. In more details, when the load level is low (high) in both cases, we describe the total load level as *very low* (high). Similar considerations are valid for the intermediate cases, too.

We carry out the experiments for a sufficient number of times to get small errors representing the 95% confidence intervals. The measured confidence intervals have not been reproduced in the graphs since all of them are small and only clutter the figures.

Figure 4 displays the average percentage of energy saving. The EA algorithm achieves a maximum energy saving around 45%. Instead, the MP algorithm does not exceed 25%. The reason of this difference lies in the different consolidation strategy, ie, the MP algorithm does not take explicitly into account the power consumption of the devices.

EA improves the results of MP when the load is not high. Indeed, when the load is high, the effectiveness of consolidation is reduced in any case, because almost all servers are full. We remark that the effectiveness of a consolidation algorithm should be evaluated under low-load conditions, though simulation of a broader range of conditions is anyway necessary to evaluate QoS parameters like the acceptance rate.

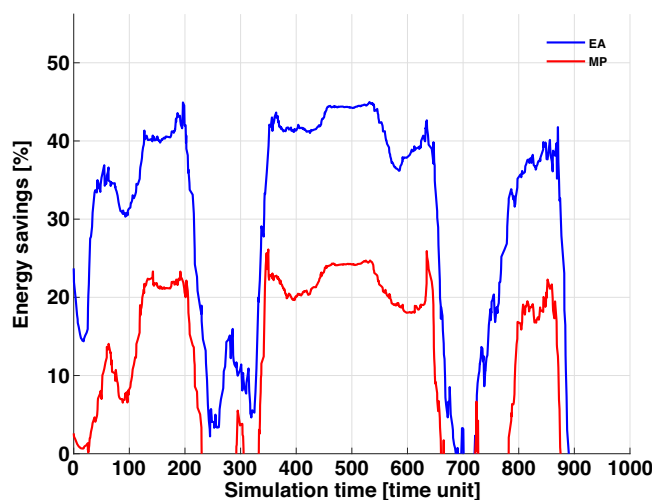


FIGURE 4 Average percentage of the energy savings. EA: Energy Aware; MP: Matching Pursuit

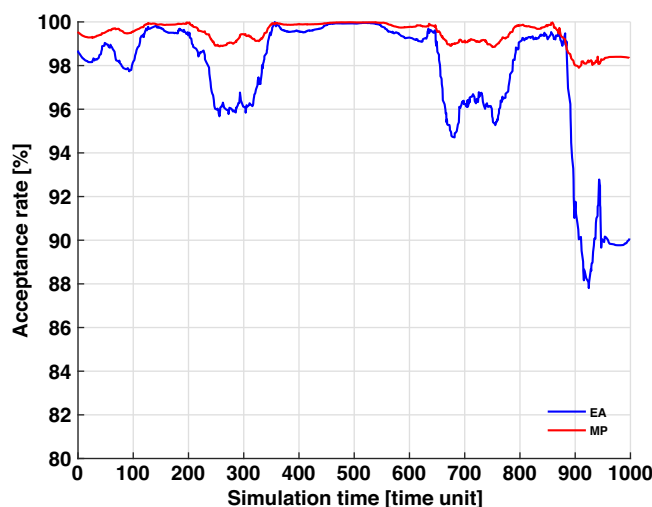


FIGURE 5 Average percentage of the accepted requests. EA: Energy Aware; MP: Matching Pursuit

The acceptance rate is shown in Figure 5. It presents excellent results for the MP solution. In this case, the acceptance rate is never less than 98%. Instead, the EA algorithm presents slightly worse performance, with acceptance rate always above 88%. However, the acceptance rate is only lower than 90% when the load is very high. We can conclude that our EA algorithm achieves a good balance between performance and QoS for large fluctuations in the workload, which is expected to be a typical condition for edge computing. In fact, it boosts large energy savings without sacrificing availability and redundancy.

6.4 | Power profiling

One of the main purposes for the implementation of an energy-efficient infrastructure (see Section 4) is fine control over power management. This includes changing the power states (eg, from active to suspend, and vice versa) and monitoring the real power consumption, so as to provide better figures to drive the consolidation procedure.

Figure 6 and 7 show real data collected in our experimental setup for servers and software switches, respectively. We call this information “power profiles” because they show how power consumption changes with different utilization levels. We observe that the power profile for the servers is slightly concave and exhibits some anomalies (Figure 6); in addition, we get different shapes when power-saving mechanisms (mainly, frequency scaling and low-power idle) are enabled or disabled. Instead, the behavior of software switches is more linear but with a very low dependency on the traffic load. We also note that power consumption is very different for heterogeneous devices, so it is important to take these differences into account in the consolidation algorithms.

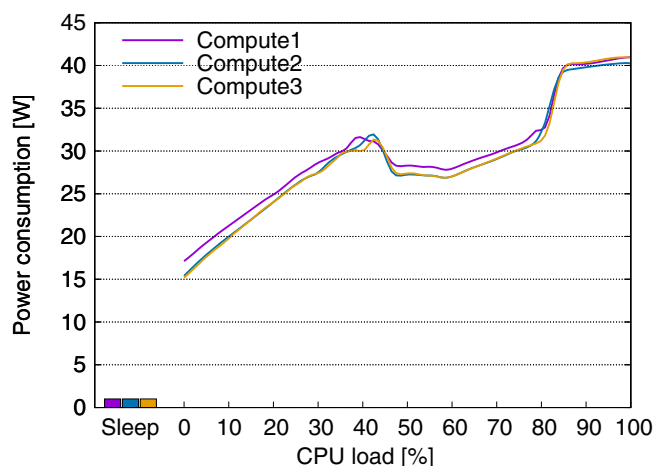


FIGURE 6 Computed power profiles for servers

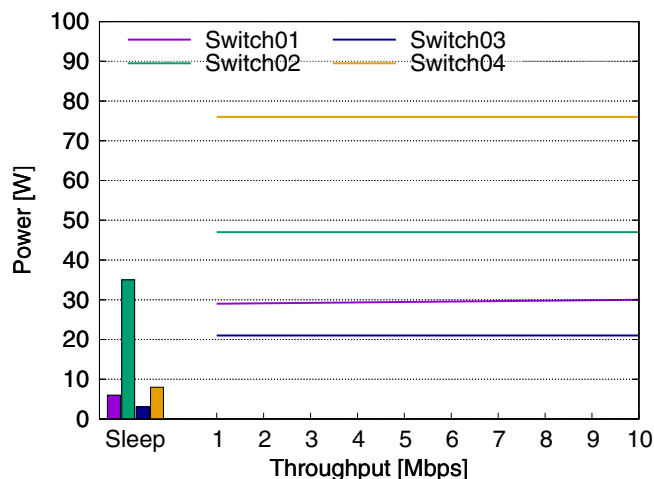


FIGURE 7 Computed power profiles for software switches

Finally, we also plotted power consumption in sleep state; the large difference from idle figures (corresponding to 0 to 1% utilization) motivates the effectiveness of consolidation.

7 | CONCLUSIONS

In this paper, we have presented a novel approach to energy efficiency in virtualized infrastructures, leveraging more context information. The main achievements are, ie, (i) the definition of an enhanced consolidation algorithm, which takes into account more context parameters (mostly related to QoS aspects) than any other similar existing proposal in the literature and (ii) the implementation of a real testbed, which enhances OpenStack with additional QoS/energy features and demonstrates the feasibility of our paradigm.

The main innovation of our work is a paradigm that allows to preprovision virtual resources and “freeze” them if unused, so as to enable more effective workload consolidation and energy saving in the underlying infrastructure. Basically, we have introduced a novel abstraction between cloud users and infrastructure providers that balances opposite needs in terms of service resilience and availability versus energy efficiency.

We envision two main benefits from our approach. Infrastructure providers can achieve better energy saving, hence offering lower rates for unused resources. Cloud users can provision in advance more resources with minimal cost, so they are able to quickly respond to unexpected events (workload variations and failures).

ACKNOWLEDGEMENT

This work was supported in part by the European Commission under the projects ARCADIA (contract 645372) and MATILDA (contract 761898).

ORCID

Matteo Repetto  <http://orcid.org/0000-0001-8478-2633>

REFERENCES

- Soldani D, Manzalini A. Horizon 2020 and beyond: On the 5G operating system for a true digital society. *IEEE Veh Technol Mag*. 2015;10(1):32-42.
- 5G empowering vertical industries. 5G-PPP, ERTICO, EFFRA, EUTC, NEM, CONTINUA and Networld2020 [ETPOnline]. 2016. https://5g-ppp.eu/wp-content/uploads/2016/02/BROCHURE_5PPP_BAT2_PL.pdf. White paper.
- Bolla R, Khan R, Repetto M. Assessing the potential for saving energy by impersonating idle networked devices. *IEEE J Sel Areas Commun*. 2016;34(5):1676-1689.
- Bonefeld-Dahl C. Let us not put high-performance servers into a permanent idle state! European Energy Innovation, Summer; 2017.
- Kanapram D, Lamanna G, Repetto M. Exploring the trade-off between performance and energy consumption in cloud infrastructures. Paper presented at: 2nd IEEE International Conference on Fog and Edge Mobile Computing (FMEC 2017); 2017; Valencia, Spain.
- Barroso LA, Hölzle U. The case for energy-proportional computing. *Computer*. 2007;40(12):33-37.
- Hammadi A, Mhamdi L. A survey on architectures and energy efficiency in data center networks. *Comput Commun*. 2014;40:1-21.
- Bolla R, Bruschi R, Davoli F, Cucchietti F. Energy efficiency in the future internet: a survey of existing approaches and trends energy-aware fixed network infrastructures. *IEEE Commun Surv Tutor*. 2011;13(2):223-244.
- Gunaratne C, Christensen K, Nordman B. Managing energy consumption costs in desktop PCs and LAN switches with proxying, split TCP connections, and scaling of link speed. *Int J Netw Manag*. 2005;15(5):297-310.
- Beloglazov A, Buyya R. OpenStack Neat: a framework for dynamic and energy-efficient consolidation of virtual machines in OpenStack clouds. *Concurrency Computat Pract Exper*. 2015;27(5):1310-1333.
- Geronimo GA, Werner J, Westphall CB, Westphall CM, Defenti L. Provisioning and resource allocation for green clouds. Paper presented at: The 12th International Conference on Networks (ICN 2013); 2013; Seville, Spain.
- Heller B, Seetharaman S, Mahadevan P, et al. Elastictree: Saving energy in data center networks. In: Proceedings of the 7th USENIX Conference on Networked System Design and Implementation (NSDI); 2010; San Jose, CA.
- Wang L, Zhang F, Vasilakos AV, Hou C, Liu Z. Joint virtual machine assignment and traffic engineering for green data center networks. *ACM SIGMETRICS Perform Eval Rev*. 2013;41(3):107-112.
- Fang W, Liang X, Li S, Chiaraviglio L, Xiong N. VMPlanner: optimizing virtual machine placement and traffic flow routing to reduce network power costs in cloud data centers. *Comput Netw*. 2013;57(1):179-196.
- Shirayanagi H, Yamada H, Kono K. Honeyguide: A VM migration-aware network topology for saving energy consumption in data center networks. Paper presented at: IEEE Symposium on Computers and Communications (ISCC); 2012; Cappadocia, Turkey.

16. Carrega A, Repetto M. Exploiting novel software development paradigms to increase the sustainability of data centers. In: Proceedings of the 9th IEEE/ACM International Conference on Utility and Cloud Computing (UCC 2016); 2016; Shanghai, China.
17. Ahmad F, Vijaykumar TN. Joint optimization of idle and cooling power in data centers while maintaining response time. In: Proceedings of the 15th Edition of ASPLOS on Architectural Support for Programming Languages and Operating Systems (ASPLOS XV); 2010; Pittsburgh, PA.
18. Srikantaiah S, Kansal A, Zhao F. Energy aware consolidation for cloud computing. In: Proceedings of the 2008 Conference on Power Aware Computing and Systems (HotPower'08); 2008; San Diego, CA.
19. Mishra M, Sahoo A. On theory of VM placement: Anomalies in existing methodologies and their mitigation using a novel vector based approach. Paper presented at: 2011 IEEE International Conference on Cloud Computing (CLOUD); 2011; Washington, DC.
20. Kliazovich D, Bouvry P, Khan SU. DENS: Data center energy-efficient network-aware scheduling. Paper presented at: 2010 IEEE ACM International Conference on Green Computing and Communications and IEEE/ACM International Conference on Cyber, Physical and Social Computing; 2010; Hangzhou, China.
21. Cima V, Grazioli B, Murphy S, Bohnert T. Adding energy efficiency to OpenStack. Paper presented at: 2015 Sustainable Internet and ICT for Sustainability (Sustain IT); 2015; Madrid, Spain.
22. Diouri MEM, Gluck O, Lefèvre L, Mignot JC. Your cluster is not power homogeneous: Take care when designing green schedulers! Paper presented at: 2013 International Green Computing Conference (IGCC); 2013; Arlington, VA.
23. Dargie W, Wen J. A probabilistic model for estimating the power consumption of processors and network interface cards. Paper presented at: 12th IEEE International Conference on Trust, Security and Privacy in Computing and Communications; 2013; Melbourne, Australia.
24. Kanapram D, Rapuzzi R, Lamanna G, Repetto M. A framework to correlate power consumption and resource usage in cloud infrastructures. Paper presented at: 2017 IEEE Conference on Network Softwarization (NetSoft); 2017; Bologna, Italy.
25. Duan Q, Yan Y, Vasilakos AV. A survey on service-oriented network virtualization toward convergence of networking and cloud computing. *IEEE Trans Netw Serv Manag*. 2012;9(4):373-392.
26. Contreras LM, López V, De Dios OG, et al. Toward cloud-ready transport networks. *IEEE Commun Mag*. 2012;50(9):48-55.
27. Puthalath H, Soares J, Sefidcon A, Melander B, Carapinha J, Melo M. Negotiating on-demand connectivity between clouds and wide area networks. Paper presented at: IEEE 1st International Conference on Cloud Networking (CLOUDNET); 2012; Paris, France.
28. Velasco L, Contreras LM, Ferraris G, et al. A service-oriented hybrid access network and clouds architecture. *IEEE Commun Mag*. 2015;53(4):159-165.
29. Sanhaji A, Niger P, Cadro P, Ollivier C, Beylot AL. Congestion-based API for cloud and WAN resource optimization. Paper presented at: IEEE NetSoft Conference and Workshops (NetSoft); 2016; Seoul, South Korea.
30. Mamatas L, Clayman S, Galis A. A service-aware virtualized software-defined infrastructure. *IEEE Commun Mag*. 2015;53(4):166-174.
31. Medhat AM, Thanh SQ, Covaci S, Carella G, Magedanz T. Orchestrating service function chaining in cloud environments. Paper presented at: IEEE 6th International Conference on Communications and Electronics; 2016; Ha Long, Vietnam.
32. Siracusa D, Salvadori E, Rasheed T. Edge-to-edge virtualization and orchestration in heterogeneous transport networks. Paper presented at: 2013 IEEE SDN for Future Networks and Services (SDN4FNS); 2013; Trento, Italy.
33. Open Networking Lab (ON.Lab). Introducing ONOS – a SDN network operating system for service providers. 2014. <http://onosproject.org/wp-content/uploads/2014/11/Whitepaper-ONOS-final.pdf>. White paper.
34. Ai Y, Peng M, Zhang K. Edge cloud computing technologies for internet of things: a primer. *Digit Commun Netw*. 2017. <http://www.sciencedirect.com/science/article/pii/S2352864817301335>. In press.
35. Adami D, Martini B, Sgambelluri A, et al. An SDN orchestrator for cloud data center: system design and experimental evaluation. *Trans Emerg Telecommun Technol*. 2017;28(11).
36. Orsini G, Bade D, Lamersdorf W. CloudAware: empowering context-aware self-adaptation for mobile applications. *Trans Emerg Telecommun Technol*. 2017;29(4). <https://doi.org/10.1002/ett.3210>
37. Buzzi S, Chih-Lin I, Klein TE, Poor HV, Yang C, Zappone A. A survey of energy-efficient techniques for 5G networks and challenges ahead. *IEEE J Sel Areas Commun*. 2016;34(4):697-709.
38. Bolla R, Sambolino L, Tigano D, Repetto M. Enhancing energy-efficient cloud management through code annotations and the green abstraction layer. Paper presented at: 2015 IEEE/ACM 8th International Conference on Utility and Cloud Computing (UCC); 2015; Limassol, Cyprus.
39. Farris I, Taleb T, Flinck H, Iera A. Providing ultra-short latency to user-centric 5G applications at the mobile network edge. *Trans Emerg Telecommun Technol*. 2017. <https://doi.org/10.1002/ett.3169>. In press.
40. Bolla R, Bruschi R, Davoli F. Green abstraction layer (GAL): Power management capabilities of the future energy telecommunication fixed network nodes. ETSI ES 203 237 version 1.1.1; 2014. Technical Report.
41. Rossignaux F, Gelas JP, Lefèvre L, de Asunção MD. A generic and extensible framework for monitoring energy consumption of OpenStack clouds. Paper presented at: 2014 IEEE 4th International Conference on Big Data and Cloud Computing; 2014; Sydney, Australia.
42. Topology and orchestration specification for cloud applications OASIS Standard. version 1.0; 2013. <http://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.pdf>
43. Quittek J. Network functions virtualisation (NFV): Management and orchestration. ETSI GS NFV-MAN 001. v1.1.1; 2014. http://www.etsi.org/deliver/etsi_gs/NFV-MAN/001_099/001/01.01.01_60/gs_nfv-man001v010101p.pdf
44. Bierman A, Bjorklund M, Watsen K. RESTCONF protocol, RFC 8040; 2017. <https://tools.ietf.org/html/rfc8040>
45. Dalvandi A, Gurusamy M, Chua KC. Time-aware VMFlow placement, routing, and migration for power efficiency in data centers. *IEEE Trans Netw Serv Man*. 2015;12(3):349-362.

46. Mahadevan P, Sharma P, Banerjee S, Ranganathan P. Energy aware network operations. In: Proceedings of the 28th IEEE International Conference on Computer Communications Workshops (INFOCOM); 2009; Rio de Janeiro, Brazil.
47. Stage A, Setzer T. Network-aware migration control and scheduling of differentiated virtual machine workload. In: Proceedings of the 2009 ICSE Workshop on Software Engineering Challenges of Cloud Computing (CLOUD); 2009; Vancouver, Canada.
48. Fang W, Liang X, Li S, Chiaraviglio L, Xiong N. VMP lanner: Optimizing virtual machine placement and traffic flow routing to reduce network power costs in cloud data centers. *Comp Net*. 2013;57(1):179-196.

How to cite this article: Carrega A, Portomauro G, Repetto M, Robino G. Balancing QoS and power consumption in programmable 5G infrastructures. *Trans Emerging Tel Tech*. 2018;29:e3425. <https://doi.org/10.1002/ett.3425>